

# Real-Life Among Us Discord-Bot – Grobkonzept

## Grundidee

Es gibt ein reales Among-Us-Spiel, das physisch in einem Gebäude oder auf einem Gelände gespielt wird. Discord dient dabei als digitale Spielzentrale. Der Bot übernimmt die Organisation, Rollenverteilung, Task-Verteilung, Fortschrittsanzeige, Leichenmeldungen, Meetings und Admin-Steuerung.

Die Spielleitung besteht aus dir und Tim. Ihr erstellt vorher die Tasks und teilt sie in drei Kategorien ein:

- short\_tasks
- medium\_tasks
- long\_tasks

Beim Start des Spiels verteilt der Bot jedem Spieler automatisch eine Mischung aus Tasks. Das Erledigen der Tasks basiert auf Vertrauen. Spieler melden ihre Tasks im eigenen Discord-Kanal als erledigt.

Ziel des Bots ist nicht, jede Aktion perfekt zu kontrollieren, sondern das Spiel übersichtlich, automatisiert und fair verwaltbar zu machen.

## Spielablauf

- Ein Admin startet eine neue Session über einen Command.
- Der Bot erstellt oder nutzt einen festen Anmeldekanal und schreibt dort eine Nachricht mit einem Button.
- Spieler klicken auf den Button und treten dadurch der Lobby bei.
- Der Bot erstellt eine Lobby für die Session.
- In der Lobby sieht man alle angemeldeten Spieler.
- Der Admin startet das Spiel.
- Der Bot verteilt Rollen.
- Der Bot erstellt für jeden Spieler einen privaten Kanal.
- Jeder Spieler bekommt im eigenen Kanal seine Rolle und seine Tasks.
- Spieler erledigen Tasks im echten Leben.
- Spieler melden erledigte Tasks im Discord-Kanal.
- Wenn eine Leiche gefunden wird, kann sie per Button oder Reaction gemeldet werden.
- Der Bot startet ein Meeting.
- Nach Diskussion und Voting wird ein Spieler rausgewählt oder es wird geskippt.
- Das Spiel läuft weiter, bis eine Siegbedingung erfüllt ist.
- Am Ende zeigt der Bot das Resultat an und löscht oder archiviert die Kanäle.

# Kanalstruktur

Es sollte pro Session eine eigene Kategorie geben.

## Beispiel:

- AmongUs Session 12
- Darin befinden sich:
- Lobby-Kanal
- Meeting-Kanal
- Admin-Kanal
- private Spielerkanäle
- Der Anmeldekanal kann einmalig erstellt werden und dauerhaft bestehen bleiben.  
Dort postet der Bot bei jeder neuen Session eine neue Join-Nachricht.

## Beispielhafte Kanäle:

- amongus-anmeldung
- amongus-lobby-12
- amongus-meeting-12
- amongus-admin-12
- amongus-timo
- amongus-tim
- amongus-kevin

Die privaten Kanäle sind nur sichtbar für den jeweiligen Spieler, die Spielleitung und den Bot.

# Session-Phasen

- Eine Session hat verschiedene Zustände.
- Lobby-Phase
- Spieler können beitreten.
- Admin kann Einstellungen prüfen.
- Admin kann Spieler entfernen.
- Admin kann Spiel starten.
- Start-Phase
- Bot sperrt den Beitritt.
- Bot verteilt Rollen.
- Bot erstellt private Kanäle.
- Bot verteilt Tasks.
- Spiel-Phase
- Spieler erledigen Tasks.
- Impostors spielen ihre Rolle.
- Leichen können gemeldet werden.
- Admin kann Status prüfen.
- Meeting-Phase
- Spiel wird pausiert.
- Alle Spieler diskutieren.
- Voting findet statt.
- Ein Spieler wird entfernt oder es wird geskippt.
- End-Phase
- Bot erkennt oder Admin bestätigt Spielende.
- Gewinner werden angezeigt.
- Rollen werden aufgedeckt.
- Kanäle werden gelöscht oder archiviert.

# Rollen

- Für die erste Version reichen wahrscheinlich zwei Rollen:
- Crewmate
- Impostor
- Crewmates
- Crewmates erhalten echte Tasks.
- Sie gewinnen, wenn alle Tasks erledigt sind oder alle Impostors rausgewählt wurden.
- Impostors
- Impostors erhalten Fake-Tasks oder dieselben Tasks wie Crewmates, aber ihr eigentliches Ziel ist es, Crewmates auszuschalten und nicht entdeckt zu werden.
- Sie gewinnen, wenn sie zahlenmässig gleich stark oder stärker als die lebenden Crewmates sind.
- Später könnte man Spezialrollen ergänzen, aber für Version 1 würde ich das nicht machen. Sonst wird das Konzept zu früh zu kompliziert.

# Task-System

Die Tasks werden vor dem Spiel von dir und Tim erstellt.

Es gibt drei Kategorien:

- short\_tasks
- medium\_tasks
- long\_tasks

## Short Tasks

Kurze Aufgaben, die schnell erledigt sind.

Beispiele:

- Gehe zu einem bestimmten Ort.
- Finde ein bestimmtes Objekt.
- Zähle etwas.
- Merke dir ein Codewort.

## Medium Tasks

Mittellange Aufgaben, die etwas mehr Zeit brauchen.

Beispiele:

- Löse ein kleines Rätsel.
- Finde mehrere Stationen.
- Hole eine Information von einem Ort.
- Kombiniere zwei Hinweise.

## Long Tasks

Längere Aufgaben, die über mehrere Schritte gehen.

Beispiele:

- Mehrteilige Rätselaufgabe.
- Suche mehrere Orte auf.
- Erfülle eine kleine Challenge.
- Sammle mehrere Informationen.

## Task-Verteilung

Beim Spielstart bekommt jeder Crewmate automatisch eine bestimmte Mischung.

Beispiel:

- 3 short tasks
- 2 medium tasks
- 1 long task

Die genaue Anzahl sollte über die Session-Einstellungen anpassbar sein.

Beispiel für kleine Runde:

- 2 short tasks
- 1 medium task
- 1 long task

Beispiel für grosse Runde:

- 4 short tasks
- 2 medium tasks
- 1 long task

## Tasks erledigen

Der Bot sendet jedem Spieler seine Taskliste in den privaten Kanal.

Bei jeder Aufgabe gibt es eine Möglichkeit, sie als erledigt zu markieren.

Das kann per Button oder Reaction passieren.

Da Tasks Vertrauenssache sind, muss der Bot nicht prüfen, ob sie wirklich gemacht wurden.

Der Bot zählt nur:

- offene Tasks
- erledigte Tasks
- Gesamtfortschritt aller Crewmates

Wichtig: Impostors sollten Fake-Tasks ebenfalls als erledigt markieren können, damit sie nicht sofort auffallen.

# Leichenmeldung

Jeder Spieler hat in seinem privaten Kanal eine Möglichkeit, eine Leiche zu melden.

Zum Beispiel:

Button: Leiche melden

Nach der Meldung fragt der Bot nach dem Fundort oder gibt eine Auswahl an Orten.

Danach passiert:

- Spiel wird pausiert.
- Meeting-Kanal wird geöffnet.
- Bot schreibt, wer die Leiche gemeldet hat.
- Bot schreibt, wo die Leiche gefunden wurde. (mal luege wie mer das lösed)
- Diskussionsphase startet.

Beispielinhalt:

- Leiche gemeldet von Timo
- Fundort: Gang beim Eingang
- Diskussionszeit: 3 Minuten
- Meeting und Voting
- Nach einer Leichenmeldung oder einem Emergency Meeting startet der Bot eine Meeting-Phase.

Ablauf:

- Diskussionsphase
- Voting-Phase
- Ergebnis
- Spiel wird fortgesetzt oder beendet
- Voting-Möglichkeiten:
- Spieler rauswählen
- Skip
- Bei Gleichstand passiert vorher definierte Regel
- Mögliche Gleichstandsregeln:
- Niemand fliegt raus.
- Es gibt eine Stichwahl.
- Admin entscheidet.
- Für Version 1 würde ich sagen: Bei Gleichstand fliegt niemand raus. Das ist einfach und nah am Original.
- Statusübersicht für Admins
- Die Spielleitung braucht jederzeit eine Übersicht.
- Im Admin-Kanal sieht man:
- Session-ID
- Spielstatus
- Spielerliste
- Rollen
- lebendig oder tot
- Taskfortschritt pro Spieler
- gesamter Taskfortschritt
- gemeldete Leichen
- Votingstatus

Beispiel:

Session 12

Status: Spiel läuft

Spieler:

- Timo – Crewmate – lebt – 3 von 6 Tasks erledigt
- Tim – Impostor – lebt – 2 von 6 Fake-Tasks erledigt
- Kevin – Crewmate – tot – 1 von 6 Tasks erledigt
- Gesamtfortschritt Crewmates:
- 11 von 24 Tasks erledigt

# Siegbedingungen

Crewmates gewinnen, wenn:

- alle echten Tasks erledigt sind
- oder alle Impostors rausgewählt wurden
- Impostors gewinnen, wenn:
- gleich viele oder mehr Impostors als Crewmates leben

Beispiel:

- 1 Impostor lebt, 1 Crewmate lebt: Impostor gewinnt
- 1 Impostor lebt, 2 Crewmates leben: Spiel läuft weiter
- 2 Impostors leben, 2 Crewmates leben: Impostors gewinnen

# Spielende

Am Ende zeigt der Bot eine Zusammenfassung.

Inhalt:

- Gewinnerteam
- Rollen aller Spieler
- erledigte Tasks
- wer wurde rausgewählt
- wer wurde eliminiert
- optional: Timeline des Spiels

Danach können die Kanäle gelöscht oder archiviert werden.



Für den Anfang würde ich machen:

- Admin klickt nach dem Spiel auf Session beenden.
- Bot zeigt Zusammenfassung.
- Bot löscht nach Bestätigung alle temporären Kanäle.
- Wichtige offene Fragen
- Spielregeln allgemein
- Wie viele Spieler braucht es mindestens?
- Wie viele Spieler sind ideal?
- Wie viele Impostors gibt es bei welcher Spielerzahl?
- Wer darf eine Session starten?
- Darf es mehrere Sessions gleichzeitig geben?
- Soll der Bot automatisch gewinnen erkennen oder entscheidet der Admin?
- Soll die Rolle eines rausgewählten Spielers angezeigt werden?
- Darf man während des Spiels reden?
- Dürfen tote Spieler noch im echten Leben herumlaufen?
- Dürfen tote Spieler noch Tasks erledigen?
- Dürfen tote Spieler in Discord noch schreiben?
- Lobby und Session
- Soll es immer nur eine aktive Session geben?
- Soll der Anmeldekanal dauerhaft existieren?
- Soll pro Session eine neue Kategorie erstellt werden?
- Sollen alte Sessions gespeichert werden?
- Sollen Kanäle automatisch gelöscht werden?
- Wie lange soll eine Lobby offen bleiben?
- Was passiert, wenn ein Spieler nach Spielstart noch beitreten will?
- Was passiert, wenn ein Spieler Discord verlässt?

## Tasks

- Wie viele Tasks bekommt jeder Spieler?
- Welche Mischung aus short, medium und long tasks ist sinnvoll?
- Sollen alle Spieler gleich viele Tasks bekommen?
- Sollen Impostors Fake-Tasks bekommen?
- Sind Fake-Tasks dieselben wie echte Tasks oder separat?
- Dürfen Tasks doppelt vergeben werden?
- Gibt es ortsgebundene Tasks?
- Soll der Bot zufällig verteilen oder soll die Spielleitung manuell zuweisen können?
- Wie verhindert man, dass ein Spieler alle Tasks sofort als erledigt markiert?
- Soll man erledigte Tasks rückgängig machen können?

## Eliminierungen

- Wie genau eliminiert ein Impostor einen Spieler im echten Leben?
- Muss das Opfer die Eliminierung im Bot bestätigen?
- Oder meldet der Impostor die Eliminierung?
- Gibt es einen Kill-Cooldown?
- Wie lang ist der Kill-Cooldown?
- Darf ein Impostor direkt nach einem Meeting jemanden eliminieren?
- Was passiert, wenn es Streit über eine Eliminierung gibt?
- Wie erfährt der Bot, dass jemand tot ist?

## Leichenmeldung

- Kann jeder eine Leiche melden?
- Können tote Spieler Leichen melden?
- Muss ein Fundort angegeben werden?
- Gibt es eine Liste mit möglichen Fundorten?
- Soll die Meldung anonym sein oder mit Name?
- Wird das Spiel automatisch pausiert?
- Wer darf nach der Meldung noch Tasks erledigen?

## Emergency Meetings

- Soll es Emergency Meetings geben?
- Wie viele Emergency Meetings pro Spieler?
- Wo kann man ein Emergency Meeting auslösen?
- Direkt im Bot oder nur an einem realen Ort?
- Gibt es nach einem Meeting einen Cooldown?

## Voting

- Wie lange dauert die Diskussion?
- Wie lange dauert das Voting?
- Wird geheim oder offen gevotet?
- Kann man Skip wählen?
- Was passiert bei Gleichstand?
- Kann der Admin ein Voting abbrechen?
- Kann ein Spieler seine Stimme ändern?
- Wird angezeigt, wer wen gewählt hat?

## Admin-Funktionen

Welche Admin-Rolle darf den Bot bedienen?

- Braucht es mehrere Spielleiter?
- Darf ein Admin manuell Spieler töten?
- Darf ein Admin Tasks bearbeiten?
- Darf ein Admin Rollen neu verteilen?
- Darf ein Admin ein Meeting erzwingen?
- Darf ein Admin das Spiel pausieren?
- Darf ein Admin Spieler entfernen?

## Fairness und Anti-Chaos

- Was passiert, wenn jemand absichtlich lügt?
- Wie streng sollen Regeln kontrolliert werden?
- Gibt es eine Spielleitung vor Ort?
- Wie verhindert man, dass private Kanäle gezeigt werden?
- Dürfen Spieler Screenshots zeigen?
- Dürfen Spieler ihr Handy offen herumtragen?
- Dürfen Spieler in Discord miteinander schreiben?

## Technik allgemein

- Welche Programmiersprache wird verwendet?
- Soll zuerst ein schneller Prototyp entstehen?
- Soll später ein Webinterface dazukommen?
- Soll eine Datenbank genutzt werden?
- Reicht für den Anfang lokale Speicherung?
- Wo wird der Bot gehostet?
- Soll der Bot dauerhaft online sein?
- Soll es Logs geben?
- Sollen Sessions nachträglich auswertbar sein?

## Sinnvolle Version 1

Für eine erste Version würde ich das Projekt bewusst klein halten.

Version 1 sollte können:

- Session erstellen
- Join-Button posten
- Spieler in Lobby sammeln
- Lobby anzeigen
- Spiel starten
- Rollen automatisch verteilen
- Tasks aus short, medium und long tasks verteilen
- private Spielerkanäle erstellen
- Rolle und Tasks senden
- Tasks als erledigt markieren
- Leiche melden
- Meeting starten
- Voting durchführen
- Spielstatus für Admin anzeigen
- Spiel beenden
- temporäre Kanäle löschen
- Noch nicht nötig für Version 1:
- Webinterface
- QR-Code-Tasks
- automatische Ortserkennung
- komplexe Spezialrollen
- mehrere parallele Sessions
- Statistiken über viele Spiele hinweg
- sehr komplexe Task-Editoren
- Empfohlene Grundentscheidung
- Ich würde das Projekt so aufbauen:
- Der Bot verwaltet das Spiel, aber die echte Spielleitung bleibt bei euch.
- Der Bot entscheidet und kontrolliert nicht alles selbst, sondern hilft euch, Chaos zu vermeiden.
- Tasks sind Vertrauenssache.
- Eliminierungen und Streitfälle entscheidet die Spielleitung.

Der Bot übernimmt nur die saubere Struktur:

- Wer spielt mit?
- Wer hat welche Rolle?
- Wer hat welche Tasks?
- Wer lebt noch?
- Wann ist Meeting?
- Wer wurde rausgewählt?
- Wann ist das Spiel vorbei?

Damit bleibt das Projekt realistisch und trotzdem ziemlich cool.